

회의록 (2026-05-27)

1. 회의 개요

- 일시: 2026-05-27 (세부 시간 확인 필요)
- 형태: 대면 + 비대면 (약 79명 참여 언급)
- 주요 목적: IT 리터러시 강의(마지막 파트) 및 실습 환경 준비(깃 설치), 운영체제/프로세스 등 CS 기초 개념 교육
- 발언자 구분(추정)
 - 강사(진행자): 대부분 설명 및 지시
 - 수강생: 설치/화면 확인 질문, 응답(“오케이”, “출석” 등)

2. 공지 및 운영 사항

2.1 AI 녹음/요약 도구 소개

- 강사가 실시간 음성 녹음 및 자동 요약 기능이 있는 도구를 소개.
- 강의 기록을 주차별로 정리해 업데이트 형태로 제공 계획(주 1회 수준 언급).
- 도구는 비용 부담이 있어 강사가 강제 추천은 하지 않으며, 테스트 후 판단 요청.

2.2 깃(Git) 설치 및 과제 공지

- 미션: 오전 중 Git 설치 및 연동 완료 요청.
- 전제 조건:
 - GitHub 회원가입 필수
 - Git 설치 필요
- 설치 안내:
 - Windows: 설치 파일 실행 → 기본값 유지, 관리자 권한 실행 권장
 - macOS: Homebrew 또는 안내 문서 따라 설치, 필요 시 Xcode Command Line Tools 설치(시간 소요 가능)
- 설치 확인:
 - 터미널/검색창에서 git 입력 시 정보 출력되는지 확인(캡처 예시 제공 언급)
- 문제 발생 시:
 - 강사에게 즉시 보고(대면/비대면)
 - 디스코드에 오류 상황 공유, 강사가 오후 접속해 지원 가능 언급

3. 강의 내용 요약 (IT 리터러시/현업 용어 • 노하우)

3.1 버전 컨트롤(Version Control)과 Git

3.1.1 버전 컨트롤의 의미와 중요성

- 소스코드는 매일 업데이트되므로 다음을 기록/관리해야 함:
 - 어떤 변경을 했는지
 - 어떤 코드가 추가/수정됐는지
 - 오류 발생 및 대응 이력
- 문제 발생 시 이전 버전으로 롤백 가능해야 하므로 필수 기본기.
- “업데이트 노트(패치노트)”처럼 문서화와 동일한 성격.

3.1.2 Git 사용 범위와 예외

- 일반 IT 서비스 기업에서 대중적으로 사용.
- 방산/은행 등 보안 민감 대기업은 GitHub 사용을 제한하고 자체 버전관리 시스템을 쓰는 경우가 있음(소스 외부 반출 우려).
- 대표 서비스 언급: GitHub(마이크로소프트 인수 언급), Bitbucket(강사는 사용 경험 없음)

3.1.3 팀 프로젝트 대비

- 향후 팀 프로젝트에서 협업 도구로 중요.
- 기본 명령어 숙지 및 에러 대응력 필요.
- 소스 업로드/다운로드로 집-수업 환경 연동하는 습관 강조.

3.2 기획/플로우차트: “사용자 불편 최소화”가 본질

3.2.1 플로우차트와 핵심 로직 설계

- 예시: 상품 접속 → 구매 → 배송 로직
- 설계 과정에서 “배송 정보는 어디서 획득?” 같은 질문이 발생 → DB/테이블 구조 이해 필요.
- 해결 예: 구매 프로세스에 로그인 단계 추가 → 회원 정보 기반으로 배송정보 자동 설정.

3.2.2 UX 관점 핵심 메시지

- “사용자가 귀찮으면 서비스는 안 쓴다”
- 입력 횟수/중복 입력 최소화가 핵심.
- 보안 요구와 사용자 편의의 충돌 지점에서 서비스 경쟁력이 결정됨.
- 사례 언급: 송금 서비스 편의성 개선(토스 초기 사례를 예로 들)

3.2.3 프로젝트 결과를 설명하는 관점

- “잘 돌아간다”는 기본값.
- 취업/면접에서 중요한 포인트:
 - 사용자 편의성이 얼마나 개선됐는지
 - 업무 효율이 얼마나 향상됐는지
- 기술 스택 나열은 경영진/면접관 관심사와 다를 수 있음(기본으로 간주).

3.2.4 제작 도구 흐름(강사 제시)

- 플로우차트 → 화면 페이지 → API 설계서 → 개발 (반복)
- 화면 설계 도구: Figma 강조
- 직무별 포인트:
 - 개발자: 개발 역량 중심(피그마는 우선순위 낮음)
 - 기획/분석: 피그마 및 설계 문서 역량 중요

3.3 오픈소스(Open Source)와 취업 전략(개발자 중심)

3.3.1 오픈소스 개념 및 라이선스

- 공개 코드 기반으로 사용/커스터마이징 가능.
- 단, 라이선스 조건 존재: 최소한 출처/저작권 표기 등 준수 필요.

3.3.2 오픈소스 참여의 의미

- 다수가 기여하는 프로젝트(예: Pandas 기여자 수 언급)를 예로 들.
- 개발자에게는 오픈소스 기여가 검증 수단일 될 수 있음(프로필/기여 기록).

3.3.3 포트폴리오에 대한 강사의 견해(발언 요지)

- 단순 CRUD/게시판 수준 프로젝트는 차별성이 약해졌다는 의견.
- 개발자 취업 경쟁력 방향:
 - 오픈소스 공개 및 기여, 또는
 - 특허/논문화 등 외부 검증 요소
- 오픈소스의 필수 동반 요소:
 - README/매뉴얼 문서 작성
 - 문서가 영어일 경우 영어 역량도 간접 증명 가능

3.3.4 팀원 모집/프로젝트 시작 시점

- 아이디어가 있으면 디스코드 등에 공개해 팀원 리크루팅 권장.
- “늦게 시작한 팀 vs 빨리 시작한 팀” 성과 차이 강조.

4. 운영체제/하드웨어 기초 강의

4.1 하드웨어를 알아야 하는 이유

- AI/LLM 확산으로 CPU/GPU/메모리 수요 증가 → 클라우드 비용 상승 가능성.
- 로컬 LLM 구축/최적화 시 하드웨어 이해가 프롬프트/튜닝/운영 효율에 영향.

4.1.1 메모리(RAM) 중요성

- LLM 작업은 메모리 소모가 크며, 학습/추론에서 특히 중요하다고 강조.
- 개인 장비 권장(강사 의견):
 - 최소 RAM 32GB, 가능하면 64GB
 - 노트북은 비싸므로 데스크탑 투자 + 원격 접속 전략 제안

4.2 운영체제(OS) 개념과 기능

4.2.1 OS 정의(강사 설명 요지)

- 하드웨어 포함 자원을 제어하고 프로그램 실행을 지원하는 시스템 소프트웨어.

4.2.2 OS별 특성(요지)

- Windows: 일반 사용자 중심, 개발에는 제약이 있을 수 있음(지원 차이 언급)
- Linux: 서버 목적 중심(“서버용”으로 요약), 개발/배포 환경에 많이 사용
- macOS: Unix 계열로 Linux와 유사한 면이 있으나 OS는 다름

4.2.3 OS 기능 4가지(강사 강조)

- 프로세스 관리
- 메모리 관리
- 파일 시스템 관리
- 네트워크 관리
- LLM/AI 서비스 구동 시 위 키워드가 실제로 자주 등장한다고 언급.

5. 프로세스/메모리/네트워크 핵심 개념

5.1 프로세스와 스레드

- 프로세스: 실행 중인 프로그램(작업 관리자 예시)
- 스레드: 프로세스 내부 실행 흐름
- 프로세스: 독립 메모리 사용
- 스레드: 프로세스 메모리 공유
- 멀티프로세싱/멀티스레딩: 병렬 작업으로 속도 개선(반복 작업/크롤링 예시)

5.2 스택/힙/코드 영역(메모리 구조)

- 스택: 지역 변수/매개변수 등 임시 데이터(함수 호출과 함께 생성/소멸)
- 힙: 동적 할당 데이터(관리 실패 시 메모리 누수 위험)
- 코드 영역: CPU가 실행할 명령(컴파일된 코드)
- 컴파일: 코드를 기계가 이해할 수 있도록 변환하는 과정(기계어로 변환 언급)

5.3 포트(Port)와 충돌 개념

- URL 구조에서 포트 위치 설명(기본은 숨겨져 있음).
- 예시: Jupyter Notebook 기본 포트(8888) 언급, 이미 사용 중이면 충돌 발생.
- 일부 프로그램은 충돌 시 새 포트 자동 할당.

5.4 컨테이너(도커)와 쿠버네티스(개념 연결)

- 컨테이너: OS에 관계없이 동일 개발 환경을 묶어 배포/이식 용이.
- 보안/운영상 이유로 서비스를 컨테이너로 분리하는 것이 일반적.
- 컨테이너가 많아지면 통합 관리 필요 → 쿠버네티스로 연결.
- 강사 강조:
 - 명령어 암기보다 리눅스/OS 원리(프로세스/포트 등) 이해가 우선.
 - “쿠버네티스만 외우는 학습”은 면접에서 한계가 있다고 언급.

6. 파일/디렉터리/마운트 개념

6.1 파일 메타데이터

- 파일명, 유형, 위치, 크기, 권한, 생성/수정 시간.
- 권한(permissions) 관리가 중요하며 chmod 개념을 예로 들(상세는 자가 학습 권고).

6.2 디렉터리(폴더)

- 디렉터리 = 파일 정보를 담은 특수 파일(수강생 이해 편의상 “폴더”로 안내)

6.3 링크(바로가기 개념)

- Windows의 바로가기처럼 원본과 별개로 접근 경로를 제공.
- Linux의 심볼릭 링크/하드 링크 개념을 “바로가기”에 비유.

6.4 마운트(Mount)

- 물리 장치를 특정 디렉터리에 연결해 OS가 접근 가능하게 만드는 작업.
- 예: 구글 코랩에서 구글 드라이브 연동 시 “마운트” 개념 활용.

7. 실습: Git Bash로 리눅스 명령어 연습

- Git 설치 목적 중 하나: Windows에서도 Git Bash로 리눅스 명령어 연습 가능
- 실습 절차(강사 시연):
 - a. 바탕화면 우클릭 → 추가 옵션 표시 → Open Git Bash Here
 - b. 폴더 생성: mkdir newfolder
 - c. 폴더 이동: cd newfolder (탭 자동완성 사용)
 - d. 파일 생성: touch 2026-05-27.txt (예시)
 - e. 목록 확인: ls
- 강사 강조:
 - 초반에는 복사/붙여넣기 지양, 직접 타이핑으로 익숙해질 것.

8. 기타 공유(시설/일상 관련)

- 싱크대에 비치된 주방제비/수세미는 강사가 개인적으로 가져다 둔 것으로, 리필은 자동 제공되지 않음.
- 점심 식사 중 작년 수강생을 우연히 만났다는 짧은 공유.

9. 결정 사항 및 액션 아이템

- (전원) 오전 중 Git 설치 및 GitHub 연동 완료
 - 완료 여부 확인은 “안 되는 사람 중심”으로 지원
- (문제 발생자) 설치 오류 시 디스코드로 상황 공유
 - 강사가 오후에 확인/지원 가능
- (전원) Git Bash에서 리눅스 기본 명령어(mkdir, cd, touch, ls) 타이핑으로 반복 연습

- (희망자) 오픈소스 참여/기여 및 README 문서화 방식 사전 탐색

- (강사, 확인 필요) 휴고(Hugo) 관련 설치/배포 가이드 블로그 링크 공유(점심시간 공유 언급)

10. 확인 필요(불명확/추가 확인 사항)

- AI 녹음/요약 도구의 정확한 제품명/요금제
- Git 설치 “오전 중 완료”의 정확한 데드라인 시간
- 디스코드 채널/공지 위치(링크/채널명)
- “42트/42트”로 표기된 대상의 정확한 의미(플랫폼/사이트/교육 시스템 여부)